

PRD v2 — Self-Improving Agent Memory & Decision System (AgentMem)

1. Overview

Product Name

AgentMem — Self-Improving Agent Memory & Decision System

Problem Statement

Current agent memory systems (Vector-RAG, chain-based updates, monolithic embeddings) fail to scale due to:

- $O(N)$ memory traversal and retrieval
- Semantic interference and hallucinations
- Lack of auditability and causality
- No principled self-improvement loop

AgentMem addresses this by combining **structured memory**, **bounded retrieval**, and **policy-driven self-improvement**, while maintaining full traceability of decisions and human interactions.

2. Goals

G1. Scalable Long-Term Memory

- Support millions of facts without retrieval degradation
- Guarantee bounded retrieval complexity
- Avoid context bloat in LLM inference

G2. Decision-Centric Intelligence

- Store not just facts, but decisions, rationales, and outcomes
- Enable replayable and explainable decision traces

G3. Self-Improvement Without Instability

- Improve retrieval, storage, and decision quality over time
- Avoid end-to-end RL through memory or live inference

G4. Human Trust & Control

- Enable humans to inspect, correct, and guide memory
- Preserve provenance and causality

G5. Multimodal & Real-World Readiness

- Support text, images, time-series, and events
 - Handle domains like markets, policy analysis, and planning
-

3. Non-Goals

NG1. End-to-End Differentiable Memory

- No neural memory that replaces symbolic structure
- No gradient flow through the memory store

NG2. Fully Autonomous Agents

- Human oversight is required for high-impact decisions
- No unsupervised self-modification

NG3. General World Models

- System focuses on task- and entity-centric reasoning
- Not attempting full environment simulation

NG4. Real-Time RL at Inference

- Learning occurs offline or asynchronously
 - Inference-time behavior remains deterministic + bounded
-

4. Target Users

- AI / ML Engineers building agent systems
- Research teams exploring agent memory & reasoning

- Decision-support systems (markets, compliance, ops)
 - Internal tooling requiring explainability
-

5. Functional Requirements

FR1. Canonicalization

- Convert raw multimodal inputs into structured entities and relations
- Attach confidence, timestamps, and sources

FR2. Structured Memory

- Entity-centric graph with typed edges
- Episodic memory for raw traces

FR3. Candidate Recall Engine

- Bounded recall via graph traversal + soft clusters
- Tunable depth and breadth

FR4. AgentMem Controller

- Policy over memory actions (store, retrieve, forget, link)
- State-aware and reward-driven

FR5. Context Trace Storage

- Persist full decision traces
- Enable replay and audit

FR6. Human-in-the-Loop

- Accept corrections, approvals, and feedback
 - Store human inputs as memory objects
-

6. Non-Functional Requirements

NFR1. Performance

- Retrieval latency < 100ms for 99p
- Memory growth linear in facts, not interactions

NFR2. Reliability

- Deterministic retrieval for identical queries
- No silent memory corruption

NFR3. Observability

- Full logging of memory operations
- Metrics for retrieval quality and memory health

NFR4. Safety

- Bounded action space for AgentMem
 - Human gating for irreversible actions
-

7. Success Metrics

Memory Metrics

- Avg. retrieval size per query
- Memory growth rate vs. signal retention
- Graph connectivity entropy

Decision Metrics

- Outcome success rate
- Calibration (confidence vs. accuracy)
- Regret vs. baseline heuristics

Learning Metrics

- Improvement in reward over time
- Reduction in irrelevant retrievals
- Stability of AgentMem policy

Human Metrics

- Correction rate
- Trust / approval ratio

- Time-to-audit per decision
-

8. Risks & Mitigations

R1. Memory Bloat

Risk: Excessive fact storage

Mitigation: AgentMem storage thresholds, decay, human pruning

R2. Retrieval Noise

Risk: Over-broad candidate recall

Mitigation: Hard recall budgets, reward shaping

R3. Feedback Loops

Risk: Self-reinforcing incorrect beliefs

Mitigation: Human validation, offline-only learning

R4. Policy Collapse

Risk: AgentMem converges to trivial behavior

Mitigation: Entropy regularization, policy constraints

R5. Human Overhead

Risk: Excessive manual intervention

Mitigation: Escalation thresholds, selective review

9. Rollout Plan

Phase 1 — MVP (Weekend Build)

- Graph memory + episodic logs
- Heuristic AgentMem
- Manual evaluation

Phase 2 — Learning

- Bandit-based AgentMem
- Reward from outcomes + human feedback

Phase 3 — Advanced Optimization

- GRPO-style offline updates
 - Cross-task policy generalization
-

10. Out of Scope (Explicit)

- Fully autonomous trading
 - Real-time policy learning
 - Black-box neural memory
-

11. Summary

AgentMem v2 is designed to be:

- **Scalable** (bounded memory + retrieval)
- **Auditable** (context traces, provenance)
- **Self-improving** (policy learning without instability)
- **Human-aligned** (oversight by design)

This PRD prioritizes **correctness, trust, and learning efficiency over raw agent autonomy**.

12. Technical Architecture Overview

AgentMem is composed of four strictly separated layers:

1. **Semantic State & Canonicalization Layer**
2. **Structured Memory Substrate**
3. **Decision & Reasoning Layer**
4. **AgentMem Control & Learning Layer**

This separation ensures learning never corrupts memory, and memory never destabilizes learning.

13. Semantic State & Canonicalization Layer

Purpose

Convert raw multimodal inputs into **low-entropy semantic state** prior to any memory write.

Design (VL-JEPA-Inspired)

- Predicts *task-relevant meaning*, not tokens
- Joint latent space across text, images, charts, and events
- Acts as a semantic denoising autoencoder

Output

- Semantic state vector
- Entity candidates
- Predicate–object hypotheses
- Confidence + provenance metadata

This layer dramatically reduces memory pollution and paraphrase noise.

14. Structured Memory Substrate

14.1 Entity-Centric Graph Memory

- Nodes: entities (people, assets, concepts)
- Edges: typed relations with timestamps, confidence, decay
- Complexity: $O(\log N)$ updates, bounded k-hop traversals

14.2 Episodic Trace Store

Stores raw interaction traces:

- Inputs
- Retrieved memory IDs
- Reasoning summaries
- Decisions
- Human interactions

Used for auditing, replay, and offline learning.

14.3 Decision Memory

Every committed decision is stored with:

- Action taken
 - Supporting memory IDs
 - Confidence
 - Observed outcome (when available)
-

15. Candidate Recall Engine

Function

Produce a bounded, high-recall candidate set for reasoning.

Retrieval Mix

- Deterministic graph traversal
- Soft semantic cluster expansion
- Temporal filtering

Hard limits enforce $O(\log N)$ retrieval complexity.

16. Reasoning & Decision Layer

- Operates only on retrieved memory IDs
- Produces decision candidates, confidence, and rationale summaries
- Cannot write to memory directly

This guarantees separation between reasoning and persistence.

17. Context Trace & Provenance System

Every decision generates a **Context Trace**:

- Input snapshot
- Semantic state summary
- Retrieved memory IDs
- Reasoning summary

- Human feedback
- Final action
- Outcome

Context traces enable explainability, trust, and learning.

18. AgentMem Controller (Self-Improvement Core)

Role

A learned policy that controls *how memory is used*, not what memory contains.

Action Space

- store / suppress
- retrieve(depth, breadth)
- link entities
- decay / forget
- defer

State Signals

- Task type
- Retrieval effectiveness
- Memory growth metrics
- Human feedback

AgentMem actions are schema-constrained and safe by construction.

19. Learning & Optimization Loop

Phase 1: Heuristics

- Static thresholds
- TTL decay
- Fixed retrieval budgets

Phase 2: Bandits

- Reward = outcome quality – retrieval cost
- Learns retrieval breadth and storage selectivity

Phase 3: Offline GRPO / RLVR

- Optimized over stored context traces
 - No gradients through memory or live inference
 - Stable and auditable learning
-

20. Engineering Implementation Notes

Storage

- Graph: Neo4j / SQLite adjacency lists
- Traces: append-only log (Parquet / JSONL)

Indexing

- Embeddings: FAISS
- Clustering: HDBSCAN / incremental k-means

Services

- Memory service (CRUD + traversal)
 - AgentMem policy service
 - Trace logging service
-

21. Key Engineering Invariants

- Memory is deterministic
- Learning is offline
- Retrieval is bounded
- Decisions are replayable

Violating any invariant is considered a system bug.

22. Final Principle

**Semantic compression before storage. Deterministic structure after storage.
Learned control at the boundaries.**